10. A* Search Algorithm

```
import heapq

def a_star_search(graph, heuristics, start, goal):
    # Priority queue stores tuples: (f_score, g_score, current_node, path)
    open_list = [(heuristics[start], 0, start, [start])]
    visited = set()

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        if current == goal:
            return path, g

        if current in visited:
            continue
        visited.add(current)

        for neighbor, weight in graph.get(current, []):
            if neighbor not in visited:
                g_new = g + weight
                f_new = g_new + heuristics.get(neighbor, 0)
                heapq.heappush(open_list, (f_new, g_new, neighbor, path + [neighbor]))

    return None, float('inf')

if __name__ == "__main__":
    # Graph layout with edge weights
    graph = {
        'A': [('B', 1), ('C', 4)],
        'B': [('D', 5), ('E', 12)],
```

```
    'C': [('E', 3)],
    'D': [('E', 2)],
    'E': []
}
# Heuristic values (straight-line distances to target 'E')
heuristics = {'A': 7, 'B': 6, 'C': 2, 'D': 1, 'E': 0}

path, cost = a_star_search(graph, heuristics, 'A', 'E')
print(f"Shortest path found: {' -> '.join(path)}")
print(f"Total path cost: {cost}")
```

